

Modelling Language Acquisition in Children, using JaCaMo

Table of Contents

Codebase Description.....	2
JaCaMo Project File, child_vocabulary_acquisition.jcm.....	4
Agent Configuration Files, *.asl.....	4
Syntax Summary.....	5
Variables.....	6
Operators.....	6
Sharing Common Code.....	7
Custom Launcher & Artefacts.....	7
VocabLauncher.java.....	7
Creating Agents, createAgs().....	7
Initialising Artefacts, initialiseArtifacts().....	7
Creating Child Agents, addChildAgents().....	7
Creating Parent Agents, addParentAgents().....	8
Creating Sibling Agents, addSiblingAgents().....	8
Synchroniser Artefact.....	8
Library Artefact.....	8
Utterances Artefact.....	9
DataLogger Artefact.....	9
WordsLearnt Artefact.....	10
Organisation configuration.....	10
Structural Specification.....	10
Functional Specification.....	11
Normative Specification.....	11

Table of Figures

Figure 1: Example of a classroom, with 10 students.....	11
Figure 2: Example of a village, containing two single-parent homes.....	11

Table of Code Snippets

Code 1: Default simulation.conf contents.....	3
Code 2: Standard contents of child_vocabulary_acquisition.jcm.....	4

Index of Tables

Table 1: Files present & their purpose.....	2
Table 2: Syntax overview for beliefs, goals, and plans.....	5
Table 3: Summary of groups used within simulation.....	10

Codebase Description

This document contains a summary of the purpose of the various files found within the codebase. It should be noted that not every file present is covered in this document.

Table 1: Files present & their purpose

Filename	Language	Purpose
gradlew	Shell Script	Entrypoint of simulation for Linux & Mac computers
gradlew.bat	Windows Batch file	Entrypoint of simulation for Windows computers
build.gradle settings.gradle	-	Configures the Gradle build environment
child_vocabulary_acquisition.jcm	-	Defines the JaCaMo environment
logging.properties	-	Configures where to log runtime output from simulation
simulation.conf	TOML	Custom configuration file for defining agents
src/agt/*.asl src/agt/inc/*.asl	Jason	Agent definitions
src/env/*.java	Java	Customised simulation launcher and various artefacts
src/org/learning_envIRON.xml	XML	Simulated organisation definition

If the simulation operates correctly, it will self-terminate upon completion. In the event that this doesn't happen, the user will need to explicitly kill the simulation. If `logging.properties` configures Gradle to output a console, `Ctrl-C` must be used to kill the simulation. If, however, the GUI interface is used, there is a 'Stop' button which should be used. In the rare event that the GUI becomes unresponsive, make sure the terminal/console window has focus then use `Ctrl-C` to kill the Gradle environment.

`logging.properties`, as noted above, tells Gradle whether to create a GUI or not. To enable the GUI, uncomment the line which reads `# handlers = jason.runtime.MASConsoleLogHandler` by removing the '#', and comment out the line which reads `handlers= java.util.logging.ConsoleHandler`.

If both of these lines are uncommented, Gradle will use whichever is defined later in the file.

Two further lines which might be useful to change relate to the verbosity of the output logged to the GUI or console. These are, respectively, `jason.runtime.MASConsoleLogHandler.level = ALL` and `java.util.logging.ConsoleHandler.level = WARN`.

Other permitted values for these latter two settings are listed elsewhere within this file.

`simulation.conf` is the primary configuration file of the simulation. The default values are shown below.

```
[agents]
child = 10
max_siblings = 0
percent_single_parents = 100.0
age_start = 3
age_finish = 8
[environment.locations]
utterances = "./resources/utterances"
books = "./resources/books"
[[ses]]
name="Welfare"
qty=8767
[[ses]]
name="Working"
qty=17514
[[ses]]
name="Professional"
qty=30142
```

Code 1: Default simulation.conf contents

It is read and processed by the custom launcher class (defined in `src/env/VocabLauncher.java`).

It is heavily commented to explain the purpose of each entry, and includes the default values used by the launcher should it be unable to correctly parse this file.

All values can be left at the defaults. However, as neither a corpus of book texts nor a corpus of utterances is included with this simulation (due to licensing restrictions), the user will need to create a `resources` directory with both an `utterances` and a `books` directory within it, then place the respective corpora within these two folders, or edit the `utterances` and `books` settings in the `[environment.locations]` section to point to the correct paths where the corpora are located. Both absolute and relative paths are supported, though only relative paths have been tested.

NOTE: If the chosen location for the corpora is within the `src` directory, Gradle will copy them across to the `build` directory it generates. Therefore it is recommended to locate these outside of the directory tree of this simulation to avoid this unnecessary duplication of files.

JaCaMo Project File, `child_vocabulary_acquisition.jcm`

```
mas child_vocabulary_acquisition {
  agent classroom_teacher:teacher.asl {
    focus: classroom.library, sync::sync.synchroniser
    roles: teacher in class
  }
  workspace sync{
    artifact synchroniser: vocab.Synchroniser
    artifact datalogger:vocab.DataLogger
    artifact wordsLearnt:vocab.WordsLearnt
  }
  workspace classroom {
    artifact library: vocab.Library
  }
  workspace home_environ {
    artifact utterances: vocab.Utterances
  }
  organisation learning_environ {
    group class : classroom {
      responsible-for : readingBooksClassroom
    }
    scheme readingBooksClassroom : classroom_reading_books
    group village: village {
    }
  }
  platform: local(asych_shared,64)
}
```

Code 2: Standard contents of `child_vocabulary_acquisition.jcm`

This file, as noted in the table above broadly defines the configuration of JaCaMo.

The first stanza, which begins with `agent`, provides the definition for the classroom teacher agent. It gives the name of the agent (`classroom_teacher`), which `asl` file to read (`teacher.asl`), specifies two artefacts for the agent to focus on as well as to take on a role within the `class` group.

The next three stanzas all define different workspaces within the system, as well as several artefacts which are to be instantiated within the system and made available for agents to interact with. The syntax is `artifact <name>: <java package.class name>`. Note the US-English spelling of 'artifact' rather than the UK-English 'artefact'.

The 5th stanza defines an organisation, which contains two groups and a scheme with the definitions of these being found in `src/org/learning_environ.xml`. At runtime, two groups are created within the system (`class` and `village`) which are defined in the corresponding sections of the XML file (`classroom` and `village`, respectively).

Lastly, the final stanza defines how JaCaMo should configure the environment it operates within. JaCaMo is a multi-threaded system with each agent, by default, being instantiated within individual threads. That is fine for small simulations where the number of agents is small compared to the multi-thread capabilities of the computer running the JaCaMo simulation. As it is possible to have JaCaMo instantiate hundreds of agents, this line tells JaCaMo to create a pool of threads which are to be shared by all of the agents. This may slow down operations in some circumstances, but it is a worthwhile trade-off against overloading the OS's ability to handle many hundreds of threads vying for access to the CPU.

Agent Configuration Files, `*.asl`

A full, detailed explanation of every part of the agent configuration files is neither possible nor valuable; most of the code in those files is well commented and explained in-situ. Additionally, a detailed explanation of how the agents operate will not be provided here.

The configuration files are written in a dialect of AgentSpeak, and includes many concepts familiar to most software programmers, although the syntax is somewhat esoteric.

There are 3 'types' of agent used in this simulation. These are: child, parent, and teacher.

The bulk of the work being done in this simulation is performed by the child agent(s), with controlled and co-ordinated input from the teacher and parent agents. As noted above, at the time of writing, there is only 1 teacher agent present within this simulation. The number of both child and parent agents is configurable through the `child` and `percent_single_parent` keys within `simulation.conf`.

At present there are no sub-types for any of the agents. To create additional agent types, at the very least it would be necessary to create an `.asl` file for each sub-type and edit the custom launcher class to create some agents using the new `.asl` file(s). To vary the proportions of agent types used, it would also be necessary to edit `ConfigFile.java`, to enable it to process additional key-value pairs within `simulation.conf`.

Syntax Summary

The agent configuration files contain any starting beliefs, goals, and plans the agent requires. It should be noted that some configuration of the child and parent agents is performed within the custom launcher (refer to the `addChildAgents()` and `addParentAgents()` methods of the `VocabLauncher` class, which is defined in `src/env/VocabLauncher.java`).

At the time of writing, there are no starting goals defined within any of the `.asl` files. There are, however, several allocated to agents by the launcher, and for the teacher agent goals are obtained through that agent taking on the teacher role within the class group and the associated mission that gets allocated to it.

Statements are grouped into blocks, with each being separated from subsequent statements by ';', while the block is ended with a '.'

Table 2: Syntax overview for beliefs, goals, and plans

Statement type	Example(s)	Purpose
Belief - starting	<code>students_found(0).</code> <code>words::unique_seen(0).</code>	Defines a 'truth' as the agent believes it to be
Belief – adding	<code>+new_belief(<value>);</code>	Add <code>new_belief(<value>)</code> to the agent's belief base
Belief – removing	<code>-old_belief(<value>);</code>	Remove <code>old_belief(<value>)</code> from the agent's belief base
Belief – testing	<code>?current_belief(CurrValue);</code>	Inspects the agent's belief base to see if it holds the belief <code>current_belief</code> , and that belief has one argument. If it finds such a belief, <code>CurrValue</code> will now hold the value of that argument. If multiple beliefs match, one is selected randomly. If no such belief is found, the agent tries to find a plan of the form <code>+?current_belief(VariableName)...</code>
Goal – starting	<code>!start.</code>	Adds <code>start</code> to the agent's list of intentions. Must be situated outside of a plan body.
Goal – additional	<code>!new_goal;</code>	Adds <code>new_goal</code> as a sub-goal of the current goal being attempted by the plan in which this statement is located. Additionally, the agent attempts to complete <code>new_goal</code> before continuing with the execution of the current plan.

Statement type	Example(s)	Purpose
Goal – new intention	<code>!!new_goal;</code>	Adds <code>new_goal</code> to the agent's list of intentions. Must appear within a plan. The agent may start trying to execute a suitable plan for <code>new_goal</code> in parallel to the current plan's execution.
Plan – belief added	<code>+new_belief(<value><-<context>:<body>.</code>	What to do when <code>new_belief(<value>)</code> is added to the belief base
Plan – belief removed	<code>-old_belief(<value><-<context>:<body>.</code>	What to do when <code>old_belief(<value>)</code> is removed from the belief base
Plan – goal added	<code>!new_goal(<value><-<context>:<body>.</code>	What to do when a new goal is given to the agent
Plan – goal failed	<code>-!old_goal(<value><-<context>:<body>.</code>	What to do when a goal being attempted fails to complete successfully
Plan – test goal	<code>?test_goal(<value><-<context>:<body>.</code>	What to do when a query of the belief base failed to find <code>test_goal(<value>)</code> within it (using the statement <code>?test_goal(<value>);</code> .

Within the `<body>` of plans, there are several more types of statements which can be used.

As one would expect, there are branching (`if`, `if...then`, `if...then...else`) and looping (`for(...)`, `while(...)`) statements.

The syntax of the `if()` statement will be familiar to most programmers: `if(Value==19) {...};`

There does not appear to be a `switch()` statement. Use either `if-elseif-else` ladders, or add a new sub-goal which has multiple applicable plans with differing contexts.

Similarly, the `while()` statement should also be familiar: `while(Value==true) {...};`

The syntax of the `for()` statement is more akin to a `for-each` from C/C++/C#: `for(.member(Value,Group)) {...};` and `for(.range(1,0,10)) {...};`

The remaining statements call operations made available by the various artefacts (both custom & built-in), and to interact with other agents

Variables

JaCaMo is rather different to other programming languages, when it comes to variables.

When naming a variable, it MUST start with a capital letter; `VariableName` is valid as a variable name, `variableName` is not (it, instead, represents a constant).

Operators

The basic arithmetic operations are represented by their standard forms used in most programming languages (namely, `+`, `-`, `*`, and `/`).

The standard inequality operators will also be familiar to most people, namely `>`, `<`, `>=`, `<=` for 'greater than', 'less than', 'greater than or equal to', and 'less than or equal to', respectively.

As is typical of the C-family of languages, assigning a value to a variable uses `=`, while performing an equality test uses `==`. To check if a value is not equal to another, `!=` is used, rather than C's `!=` or BASIC's `<>`.

Sharing Common Code

JaCaMo allows 1 .asl file to be include others through `{include("<filename>.asl")}`. This feature has been used to enable beliefs, goals and/or plans to be used by multiple agents, reducing the amount of code (and decreasing the troubleshooting burden). Additionally, JaCaMo provides a number of .asl files, and some of these are included within `common.asl`.

Custom Launcher & Artefacts

VocabLauncher.java

At launch, Gradle attempts to re-compile any Java files it finds within the directory tree. If that succeeds, it will attempt start the JVM using the `VocabLauncher.main()` as the entrypoint. In this method is a call to `create()`, which in-turn calls `createAgs()`. It is in `createAgs()` where the customisation of the launcher has been effected.

Whilst it would have been possible to develop this simulation without creating `VocabLauncher`, it would have required future users of the simulation to be editing `child_vocabulary_acquisition.jcm`. While relatively simple to do, it also is where several key pieces of the simulation's architecture are configured, and it was felt that the potential for erroneous configuration was too great, and would lead to ever more complex error handling by the agents themselves.

Creating Agents, `createAgs()`

There are five methods which are used to process the customisation and they are all called from `createAgs()`. The first is to read `simulation.conf`, which ensures default values are available if any of the expected keys are absent. Next the custom artefacts are initialised. The final three methods create the child agents with the required configuration, an appropriate number of parent agents, and a stub-method to handle creation of sibling agents.

Initialising Artefacts, `initialiseArtifacts()`

There are, at present, four custom artefacts required for the operation of this simulation.

To track the progress through the simulated years, and to ensure all agents act at appropriate times, a `Synchroniser` artefact has been created. It exposes a number of operations which can be called by the agents, as well as raises signals to alert agents to the new 'state' of the simulation.

As the primary purpose of this simulation is to enable the modelling of children acquiring vocabulary, two artefacts were created to represent a library of books and a collection of phrases which can be 'spoken' to the child agents. These are the `Library` and `Utterances` artefacts, respectively.

Lastly, while it is possible to use the web interface to interrogate the child agents to obtain the details of their vocabulary acquisition, it is far quicker and less error-prone to have the agents report that information to a `DataLogger` artefact, which writes out two CSV files at the completion of the simulated years.

Creating Child Agents, `addchildAgents()`

To provide the greatest flexibility as well as to improve the reliability of the results generated, it was determined that randomly allocating child agents to 'homes' was appropriate.

After the home allocations are randomised, this method creates & configures a 'template' child agent. This template agent is told to focus on the above artefacts, take on a role within the 'village' and given 2 starting beliefs & 1 starting goal.

Finally, this method creates the actual agent which will be operating within the simulation by copying the template agent and customising it by changing the agent's name and giving it a starting goal to join a specific 'home' group, before inserting the agent into the simulation.

Creating Parent Agents, `addParentAgents()`

In a similar vein as for the child agents, it was deemed appropriate to randomise the allocation of SES categories to the parent agents (with the caveat that they needed to be allocated as evenly as possible).

The number of parent agents which need to be created is dependent on both the number of child agents and on the value of the `percent_single_parent` key in `simulation.conf`.

As with creating child agents, a 'template' parent agent is created with the common elements before being copied and customised to suit for each.

Unlike the child agents, the parents are allocated to the home groups in a predictable manner, with the lower numbered homes also getting a 2nd parent agent if the `percent_single_parent` value is not 100. The 1st (or only) parent in each house is given beliefs regarding the SES category and how many parents are in the house as well as a goal to actually create the home group. The 2nd parent agent is given a goal to join the home group.

Finally, as with child agents, the parent agents are inserted into the simulation.

Creating Sibling Agents, `addSiblingAgents()`

At the time of writing, this method is a stub and does nothing.

It has been envisaged that additional children could be added to each household, to provide additional sources of vocabulary. Several ideas have been mulled over regarding how much and what types of interaction they will have with the child agents, however nothing definitive as been decided beyond a practical limit of having no more than 2 siblings per house. Even a relatively small simulation involving 30 child agents, and both 2 parents and 2 siblings per household involves 150 agents.

Synchroniser Artefact

This is responsible for tracking and advancing the simulation's state, signalling the new state to the simulation, and at the end of the simulation it co-ordinates the finalisation process which involves the child agents reporting all of the words they learned during the simulation to the `WordsLearnt` artefact before telling the `DataLogger` artefact to generate the output data files, and then finally shutting down JaCaMo and notifying the user of the names of those data files.

Library Artefact

This contains all of the books which are available to be read by the various agents within this simulation.

It is initialised by calling `loadBookTitles()` with a directory path which should contain a number of UTF-8 formatted `.txt` files. As it is anticipated that there will be a significant number of books, the initialisation routine only reads the filenames into a `HashMap` as keys with empty `String` objects as the values, instead of the potentially slow process of reading the contents of all books in to memory only to have most of that effort to not necessarily be required. The number of files present in the given directory is visible to agents via a `book_count ObservableProperty`.

When an agent wants the text of the book it calls the `get_bookByTitle()` operation, which ensures the title does exist within the dataset. If the book does exist, the next check is to see if the book's text has been loaded already. To ensure only one attempt is made to read the book's contents from the file, a `Lock` object is utilised. It is locked, then the check of the book's text is made, loading the book's text if necessary, before finally unlocking the lock. Lastly, the text is split into separate words before being sent back to the agent.

The `Lock` object is essential, as noted above, due to the multi-threaded nature of JaCaMo. Several agents could call `get_bookByTitle()` for the same book in quick succession, which could result in multiple attempts to load the book's text from the file and placing it into the `HashMap`. The `lock()` method call blocks until the lock is obtained by the calling thread, which ensures that only one thread will attempt to call `loadBook()` for any specific book.

Similarly, the `getWordCountByBookTitle()` operation exists so that an agent can ensure there are some words available to 'read' before attempting to do so. It operates in an identical manner, in that it fails out early should the book title be invalid, locking the `Lock`, ensuring the book's text is loaded, then chunking the long text string down into individual words before counting them and returning that value to the agent.

With enough iterations within one run of this simulation it is likely that a significant number of books will be read from the files. The dataset being used to develop this simulation contains over 5000 books. The default configuration for this simulation would see 6 years of time being simulated, which equates to only 2190 books (less than 50% of the dataset) at most being used, at the time of writing.

Utterances Artefact

Similarly to the `Library` artefact, an `Utterances` artefact was created to serve as a dataset of short phrases which are available to agents to use to simulate speaking to another.

Unlike the books in the `Library`, the utterances are loaded into memory in their entirety during initialisation, due to the formatting of the data. Each file on disk holds many utterances as well as potentially useful metadata which can be made available to agents.

To ensure that there is only one attempt to load the utterances, the `loadUtterances()` method first checks to see if `utterancesMap.size()` is not 0. Only then does it attempt to load the utterances and participant metadata from disk.

As the dataset is CSV-formatted, and there was no need to 're-invent the wheel', the `OpenCSV` library is utilised to parse the CSV files for reading. This library has been proven reliable at handling the vast majority of possible combinations of CSV formatting systems which could be used, and has excellent error handling functionality included. This allows us to focus on processing the data in the files.

The `loadUtterances()` method targets `.csv` files with "utterances" in the file's name, and selects just `speaker_code` and `gloss` columns to be read.

The `loadParticipants()` method also target `.csv` files, however it is only interested in those files with "participants" in their name and selects only the `role` and `id` columns.

Both of these methods call `HashMapFromCSVFiles()`, which does the heavy-lifting of actually reading in & parsing the files, and building a `HashMap` from the selected fields.

From the agents' perspective, they are able to see the number of unique 'codes' which identify the speaker of an utterance and the total number of utterances available. Should the agent wish to, they are able to get a list of the speaker codes available using the `getSpeakerCodes()` operation, which then enables them to provide a valid `speakerCode` to `getRandomUtterance()` (which uses the `speakerCode` to select an `ArrayList<String>` from `utterancesMap`). Alternatively, an agent can request a bulk lot of utterances, using the `getBulkUtterances()` operation, passing in the number of words required, and gets back both an `Array` of `String` objects and the number of words obtained (which will be no less than the number requested).

DataLogger Artefact

This is responsible for creating the two CSV files containing the pertinent output data from each execution of this simulation.

At the start of the simulation, this artefact obtains the current system date & time and stores it for later use.

It exposes one operation for agents, the `addAnnualStats()` method which is used by the child agents to report how their name, SES, age, and the number unique words the agent has seen, heard and learned to that point.

It also contains two convenience methods, allowing the user to be notified of the file names of the CSV files created.

The bulk of the code is within the `writeFiles()` method, which checks that some annual stats have been provided by the agents before attempting to write any files. It then attempts to write-out all of the annual stats which have been reported, before attempting to interrogate the `WordsLearnt` artefact to write out the list of words learned by the agents and how old each agent was when it learned the word.

WordsLearnt Artefact

Although it could easily be incorporated within the `DataLogger`'s functionality, it was decided to have a separate artefact to receive & process the lists of words learned by the agents. It exposes one operation to the agents, `addLearnedWords()`, which expects 3 arguments representing the agent's name, the agent's SES category and an Array of Objects. Each element is actually a `String` of the form `"aoa(<word>, <age>)"`.

The `DataLogger` artefact calls the `getAll()` method to receive all of the words as an `ArrayList<WordLearntRow>`, a record-type object created specifically to handle its transfer.

Organisation configuration

Moving on to how all the above mentioned groups relate and link to each other, it was evident from early on that utilising the capabilities of the Moise framework within JaCaMo would be advantageous though it is not necessary to do so; this simulation could be implemented entirely without using any part of the Moise framework.

To this end, several roles and groups have been defined. While there is no restriction on which agent can take on any given role, the roles are named in such a way as to make it obvious that certain agents should not take on certain roles (i.e. a child agent isn't expected to take on any of the adult, parent, or teacher roles, nor a parent agent take on any of the child, offspring, or student roles).

The configuration of Moise is divided into a structural, functional, and normative specifications, each of which are explained in the following sections.

Structural Specification

In the table and both figures in this section, UML notation is used to describe cardinalities; UML symbols are used in the figures to represent the various elements. Specifically, the oval symbols represent specific agents, rounded rectangles represent roles, packages represent groups, and the hexagon represents a scheme; for the various lines, black arrows represent an agent taking on a role, the hollow diamonds indicate group membership, and the dotted lines represent a link between roles. The unattached roles shown in the two figures indicate that those roles are present within the simulation but play no part in the group(s) depicted.

Table 3: Summary of groups used within simulation

Group	Roles & cardinalities	Sub-groups & cardinalities
classroom	teacher (1..1) student (1..*)	-
village	adult (1..*) child (1..*)	indv_home (1..*)
indv_home	parent (1..2) offspring (1..1) sibling (0..2)	-

The group names used in the table above are those defined in `learning_envIRON.xml`, however the names of the groups which are instantiated as specified in `child_vocabulary_acquisition.jcm` are not required to match (i.e. `classroom` is instantiated as `class`, `indv_home` is `home_1`, `home_2`, `home_3`, etc., but `village` is instantiated as `village`); see figures 1 & 2.

The `sibling` role is included in the table above to indicate where it is intended to be used although currently there is no implementation of it.

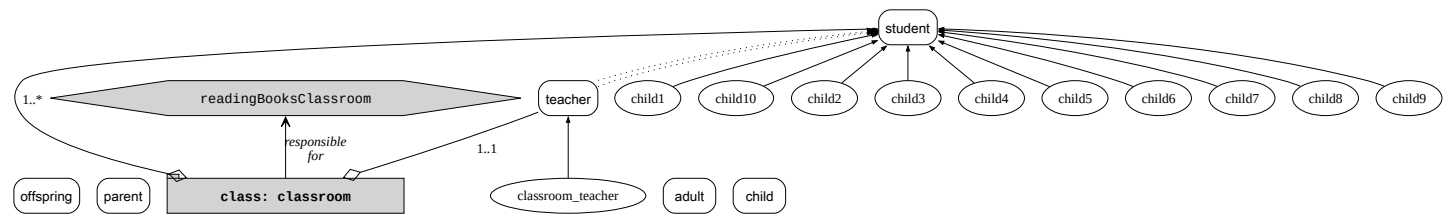


Figure 1: Example of a classroom, with 10 students

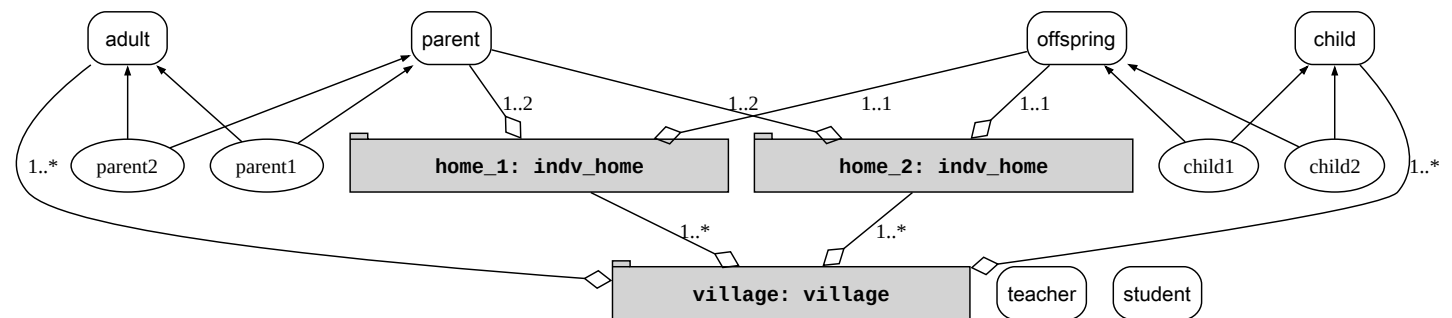


Figure 2: Example of a village, containing two single-parent homes

JaCaMo allows agents to join multiple groups concurrently, and the child agents end up joining a total of three groups, namely the `classroom`, `village`, and their individual home group. To demonstrate the flexibility of JaCaMo, the child agents are given goals of `join_school` and `join_home_group()` but allocated the role of `child` within the `village` group, within `VocabLauncher.addChildAgents()`, with plans to achieve the two goals mentioned being found in `child_common.asl` and using the `adoptRole()` operation, showing that roles can be both dynamically and prescriptively applied.

Functional Specification

The functional specification section of `learning_envIRON.xml` defines some 'business procedures', which automatically impose restrictions & requirements on the execution sequencing of some of the goals.

Although two schemes are defined only one is currently used. Schemes each contain a goal tree as well as 1 or more mission definitions.

As with group definitions & group instance names, scheme definitions and instances can have differing names. As shown in figure 1, the `readingBooksClassroom` scheme is the instantiated name of the scheme `classroom_reading_books` defined within `learning_envIRON.xml`. This scheme have one primary goal, consisting of two sub-goals which are to be completed in sequence (as specified by the `operator` attribute on the `plan` element) for the scheme to complete successfully. Multiple agents, playing multiple roles could be required to complete the goals of the scheme, however in this simulation only a teacher agent is involved.

Missions collate 1 or more goals into a collection which 1 or more roles may be tasked to attempt to achieve.

Normative Specification

Lastly, the normative specification defines any 'norms' applicable to the simulation. As noted above, these link roles to missions. As with the functional specification, there are two norms defined although only one is currently used.

The `teacherNorm` places an deontological obligation upon any agent taking on the teacher role, requiring that it must attempt the `tellStudentsToRead` mission (which is defined within the functional specification section).

The `parentNorm`, similarly, places an obligation on the parent role to achieve the `speakUtterances` mission, however as no group within the organisation is responsible for the `speak_to_child` scheme (where the `speakUtterances` mission is defined), this norm has no effect on the operation of the simulation. To bring it in to effect, one way this could be achieved is through editing the `+!create_home_group()` plan to include the `createScheme()` and `addScheme()` actions.

As noted in the Sharing Common Code section, a number of `.asl` files supplied by JaCaMo are included by the `.asl` files used by the agents. One of those supplied files contains code which makes an agent obedient to the organisation's objectives. The effect of not having the agent(s) being obedient has not been tested. Technically, all of the agents in the simulation are obedient, however only the teacher agent is actively affected through the `teacherNorm` norm and the `classroom_reading_books` scheme. If the `village` and/or `indv_home` groups were made responsible for the `speak_to_child` scheme, any agent taking on the parent role would also be affected.